L1 学习手册：测试设计方法与缺陷管理

游戏测试必备层 · 五种设计方法 + 缺陷管理 · 含游戏场景案例与练习

配套《8 周逐日执行手册》W1 至 W2 使用 ｜ 建议用时 12 至 16 小时

使用说明
第一部分 · 五种测试设计方法
第二部分 · 五种方法如何配合
第三部分 · 缺陷管理
第四部分 · 练习题与答案

使用说明

这份手册的用法和教材不同，**不要通读**。正确顺序是：

1. 先跳到每个方法的「案例」小节，只看案例，看懂这个场景在测什么
2. 再回头读「本质」和「怎么做」，把方法挂到你刚看过的场景上
3. 合上手册，自己在纸上把案例重做一遍
4. 做第四部分的练习题，对照答案找差距

第 1 步和第 2 步的顺序不能颠倒。先看现象再学定义，理解速度会快数倍；反过来则容易变成背名词。

第一部分・五种测试设计方法

五种方法不是并列关系，而是**从不同角度切同一个系统。**掌握单种不难，难的是知道面对一个功能时该用哪个、以及怎么组合。

1. 等价类划分 Equivalence Partitioning

> **本质：**把无穷多的输入切成有限几块，每块里任取一个值，测出来的效果相同。所以每块只测一个。

定义

将输入域划分为若干子集（等价类），同一子集内的任意输入对程序的作用等价。分有效等价类与无效等价类两类，前者验证正常功能，后者验证异常拦截。

什么时候用

- 输入范围很大、无法穷举（数量、金额、文本）
- 需要覆盖异常输入，但异常情况无穷多
- 作为边界值分析的前置步骤

怎么做

1. 识别所有输入项与外部条件
2. 为每个输入项划分有效等价类
3. 为每个输入项划分无效等价类
4. 为每个类取一个代表值，形成用例

案例：商城购买数量的输入框

某手游商城支持自定义购买数量，单次可购 1 至 99 个。输入域划分如下：

类别	划分	代表值	预期结果
有效	1 至 99 的整数	1、50、99	按数量扣款并发货
无效	小于 1	0、-1	拦截，提示最小购买量
无效	大于 99	100、999	拦截，提示最大购买量
无效	非整数	1.5	需确认设计：拦截还是向下取整
无效	非数字	abc、空格、中文	拦截，输入框不接受
无效	空值	空字符串	拦截，或默认为 1
无效	特殊字符	% 、' 、; 、<script>	拦截，且不得引发注入

最后一行的特殊字符不只是“能不能输入”的问题，还要验证不会引发 SQL 注入或脚本注入。这一点多数新人会漏，但它是安全测试的入口，写进用例集是明确的加分项。

常见错误

- × 只划分有效等价类，不测无效等价类，等于只测了能用的路径
- × 把 0 和 100 当成等价类代表值，但它们其实属于边界值，应该在下一步单独处理
- × 遇到设计未定义的情况（如 1.5）直接跳过。这类模糊地带恰恰是真 bug 的高发区，应该记录下来并在评审时向策划确认

2. 边界值分析 Boundary Value Analysis

> **本质**：绝大多数缺陷发生在边界上，而不是中间值上。因为边界是判断条件写错的地方。

定义

对输入或状态的边界值进行测试。取边界本身（上点）、紧邻边界的有效值（内点）、刚超出边界的无效值（离点）三组数据。

什么时候用

- 输入有明确取值范围（数量、金额、等级、时间）
- 状态有容量上限或下限（背包格数、好友上限）
- 数值比较逻辑（大于、小于、等于）

怎么做

1. 确定边界（上限、下限）
2. 取上点：边界本身的值
3. 取内点：边界内侧最近的有效值
4. 取离点：边界外侧最近的无效值
5. 对每个边界重复以上三步

案例：背包容量 200 格

背包上限 200 格，从空到满的完整边界测试点如下：

数量	类型	场景	预期结果
0	零点	空背包	显示空状态引导，不报错、不崩溃
1	内点	1 个道具	正常显示，排序正确
199	内点	差一格满	仍可正常拾取第 200 个
200	上点	正好满	拾取第 201 个时提示“背包已满”，道具不消失
201	离点	超容量	不可能通过正常操作达到，需验证扩容或系统发放时的处理

> 游戏测试的关键差异在于：**边界不只是输入框，状态本身也有边界。**血量 0 的死亡判定、金币 0 时的余额不足、等级上限后的经验处理、23:59:59 的跨天重置、扣款导致余额为负——这些都属于边界值分析的范围。只盯着输入框测边界，会漏掉游戏里最核心的一批缺陷。

> **常见错误**
> - × 只测输入框的边界，忘了状态边界（血量、金币、等级、时间）

× 只测上点不测离点，导致“超出一个”的越界情况漏测
× 没测零点。空背包、零金币、零好友这些状态最容易触发空指针或除零错误

3. 判定表 Decision Table

> **本质：**当多个条件共同决定一个结果时，用表格把全部组合列穷，避免凭感觉漏掉分支。

定义

把条件（输入）与动作（输出）的对应关系列成表格。先列条件桩与动作桩，再枚举条件的全部组合，逐列填写应执行的动作。适用于条件之间有逻辑组合的场景。

什么时候用

- 多个条件共同决定结果（如折扣 = 会员等级 × 优惠券 × 活动）
- 业务规则复杂、口头描述容易产生歧义
- 需要向策划确认“某种组合下到底该怎么处理”

怎么做

1. 列出所有条件（条件桩）
2. 列出所有可能的动作（动作桩）
3. 枚举条件的全部组合，形成规则列
4. 逐列填写应执行的动作
5. 合并等价规则，减少用例数
6. 每一条规则对应一条测试用例

案例：抽卡保底规则

规则设定：单抽或十连；90 抽内必出 SSR；十连必出至少 1 个 SR；持有保底券时优先消耗券。三个条件组合后共 8 条规则：

规则	十连	计数达 89	有保底券	用券	出 SSR	出 SR	计数重置	计数 +1
R1	否	否	否	—	—	—	—	✓
R2	否	否	是	✓	—	—	—	✓
R3	否	是	否	—	✓	—	✓	—
R4	否	是	是	✓	✓	—	✓	—
R5	是	否	否	—	—	✓	—	✓
R6	是	否	是	✓	—	✓	—	✓
R7	是	是	否	—	✓	✓	✓	—
R8	是	是	是	✓	✓	✓	✓	—

> 判定表最大的价值是**把“感觉要测很多”变成“就是这 8 条”**。三个条件如果凭感觉测，通常只会覆盖 R1 和 R8 两个极端，中间 6 条全漏。而 R3 与 R4 的区别（用券时是否仍消耗保底计数）正是最容易出 bug 的地方。

常见错误

× 只测全满足和全不满足两个极端，中间组合全漏
× 条件之间有互斥关系时没有合并规则，导致组合数量爆炸，最后放弃测试
× 没有把“计数 +1 还是重置”这类互斥动作明确写出来。两个动作同时触发是逻辑矛盾的典型征兆

4. 状态迁移法 State Transition Testing

> **本质：**把功能看成一个状态机，测的是“从哪个状态、经过什么事件、到达哪个状态”，重点是非法迁移。

定义

将被测对象抽象为若干状态与触发事件，绘制状态迁移图或迁移矩阵，覆盖全部合法迁移，并逐一验证非法迁移是否被正确拦截。

什么时候用

- 功能有明确的状态流转（任务、订单、好友申请、战斗流程）
- 状态数量有限且可枚举
- 存在“某些操作在特定状态下不应该被允许”的约束

怎么做

1. 枚举全部状态
2. 枚举全部触发事件
3. 绘制迁移矩阵（行=起始状态，列=事件）
4. 标出合法迁移与非法迁移
5. 每条合法迁移写一条用例
6. 每条非法迁移也要写一条用例，验证被正确拦截

案例：任务系统

六个状态、五个事件，构成如下迁移矩阵。表中 ✗ 表示该事件在此状态下应被拒绝——每一个 ✗ 都是一条必测用例。

起始状态 \ 事件	E1 接取	E2 达成条件	E3 交付	E4 超时	E5 放弃
S1 未接取	S2	✗	✗	✗	✗
S2 已接取	✗	S3	✗	S6	S1
S3 进行中	✗	S4	✗	S6	✗
S4 可交付	✗	✗	S5	S6	✗
S5 已交付	✗	✗	✗	✗	✗
S6 已过期	✗	✗	✗	✗	✗

> 这个矩阵共 30 个格子，其中合法迁移 6 条、非法迁移 24 条。**非法迁移才是缺陷最密集的区域**：已交付的任务能否再次接取、已过期的任务能否交付、进行中的任务能否被放弃——这些路径如果后端没有做状态校验，就会出现重复领奖、奖励刷取这类严重问题。

常见错误

× 只测合法迁移，不测非法迁移。这是状态迁移测试最常见的错误，也是最容易漏掉严重缺陷的地方
× 漏掉隐式状态。任务除了“进行中”，还有“已完成但未刷新”“跨天后未重置”，这些不在策划案里明说，但真实存在
× 没有测试状态的回退与重进：切后台、断网重连、杀进程后重新进入，状态是否保持一致

5. 场景法 Scenario Testing

> **本质：**按用户的真实使用路径把功能串起来测，验证端到端流程是否走得通，重点是备选流（异常分支）。

定义

从用户视角识别基本流（正常路径）与备选流（异常、分支路径），将二者组合成完整场景。用于验证业务流程的完整性与连贯性，是集成层面的测试方法。

什么时候用

- 需要验证端到端业务流程（充值、注册、新手引导）
- 功能之间存在依赖关系，单独测都通过但串起来会出问题
- 需要评估业务风险，确定测试优先级

怎么做

1. 识别基本流：用户最常走的那条路径
2. 识别备选流：取消、超时、失败、中断、重复
3. 将备选流插入基本流的各个节点，形成场景
4. 为每个场景标注风险等级
5. 高风险场景优先执行

案例：充值支付流程

基本流是 8 个步骤，但真正的测试价值在备选流上：

场景	路径	风险	验证重点
基本流	选商品 → 支付 → 回调 → 发货 → 到账	—	全链路走通，钻石数量正确
A1	用户取消支付	中	订单状态为已取消，不可发货
A2	支付超时（15 分钟未完成）	中	订单自动关闭，不可再支付
A3	支付成功但回调丢失	高	有主动查询补偿机制，不能丢单
A4	回调重复推送	极高	幂等处理，只能发一次货
A5	发货失败（背包满、道具不存在）	高	有重试或人工补偿，不能吞单
A6	支付过程中断网	高	重连后订单状态一致
A7	支付成功后杀进程	中	重新进入能查到订单与到账记录
A8	多端同时登录同一账号	高	订单不可重复使用

A4（回调重复推送）是充值类缺陷的头号来源，因为它直接对应“重复到账”，属于资金损失，一律定为最高优先级。备选流不是补充，**它承担了这条链路 80% 的测试价值**——基本流只要跑通一次就够，备选流每一条都可能对应一次线上事故。

常见错误

× 只测基本流（happy path）。这是新人最典型的错误，也是面试官最容易识破的地方

× 备选流写得笼统，比如只写“支付异常”，没写清是取消、超时还是回调丢失

× 没有给场景标注风险等级，导致测试执行顺序不合理，高风险场景被排在最后

第二部分 · 五种方法如何配合

真实工作中不会只用一种方法。面对一个功能，标准的推导顺序是固定的。

2.1 选择决策

如果面对的是	先用	再补
有明确取值范围	等价类划分	边界值分析
多个条件组合决定结果	判定表	等价类收敛组合数量
有状态流转	状态迁移法	每个状态内的等价类与边界值
端到端业务流程	场景法	把流程中的每个节点拆给前三种方法

2.2 综合案例：商城购买系统

假设要测一个完整的商城购买功能，五种方法的分工如下：

步骤	方法	做什么	产出
第一步	场景法	串出骨架：浏览 → 选择 → 确认 → 支付 → 到账，并补充取消、超时、失败等备选流	9 条
第二步	状态迁移法	拆解订单状态：待支付、已支付、已发货、已取消、已退款，列出全部迁移与非法迁移	14 条
第三步	等价类 + 边界值	补输入：购买数量、支付金额、账户余额	12 条
第四步	判定表	处理组合：会员折扣 × 优惠券 × 余额是否充足	8 条
合计	—	覆盖正常路径、异常路径、输入边界、业务规则组合	约 43 条

对照一下：如果凭感觉“点一遍”，通常只能覆盖 10 条左右，而且遗漏几乎全部集中在非法路径上。**43 条与 10 条的差距，不在勤奋程度，而在有没有推导方法。**这也是面试官让你现场设计用例时真正在考察的东西——他会追问“这条是怎么来的”。

第三部分 • 缺陷管理

3.1 缺陷生命周期

标准流程：`新建 → 已指派 → 修复中 → 待验证 → 已关闭`

三个分支需要理解清楚：

- **重新打开（Reopened）**：验证不通过，或修复引入了新问题。要写清重新打开的理由，不要只说“没好”
- **拒绝（Rejected）**：分四种——不修（成本高于收益）、无法复现（信息不足）、设计如此（你理解错了需求）、重复（已有同类单）
- **延期（Deferred）**：确认是缺陷，但排到下个版本。游戏行业版本节奏快，这种情况很常见，**务必记录影响版本号**，否则会在线上复现

> **被“拒绝”时的正确做法：**不要直接接受。先确认是否是自己信息不足（补日志、补录屏、补复现步骤）；如果确认是设计如此，就把这条记录为“已知设计约束”，写进用例的备注。下次评审需求时，这类约束应该提前问清楚。

3.2 严重程度与优先级

这是面试必考，也是最容易答错的一题。二者的判定主体不同：

维度	严重程度 Severity	优先级 Priority
定义	缺陷对系统的破坏力	修复的紧急程度
判定者	测试人员（技术判断）	产品经理或主程（商业判断）
依据	功能失效程度、数据影响范围	影响用户量、上线时间、修复成本

把两者交叉，会得到四种组合，每一种在真实项目里都存在：

	高优先级	低优先级
高严重	崩溃、卡死、充值不到账、存档丢失、无法登录	仅某款低端机型崩溃，该机型用户占比 1%
低严重	登录页文案错别字、首充按钮价格标错（影响品牌与收入）	边缘界面 1px 对齐问题、提示语不够友好

> **关键认知：高严重不等于高优先级。**资源有限时，公司按“影响用户量 ÷ 修复成本”排序，而不是按破坏力排序。能说清这一点，说明你理解真实的项目运作；只会说“严重就得先修”，说明你还没进过项目。

3.3 严重程度分级参考

级别	名称	判定标准
S1	致命	崩溃、卡死、数据丢失、充值异常、无法进入游戏。阻塞测试或阻塞上线
S2	严重	核心功能不可用（无法战斗、无法交易）、数值计算错误、关键流程中断
S3	一般	非核心功能异常、界面错误但不影响操作、提示信息有误
S4	轻微	文案错别字、UI 不美观、交互不够顺畅

3.4 缺陷分类

功能、性能、兼容性、UI 与交互、数值、崩溃、网络、安全、文案、资源（美术与音效）。分类的价值在于统计分布——如果某版本兼容性缺陷占比突增，说明新适配的机型有问题，这是质量趋势分析的起点。

3.5 一份合格的缺陷单包含什么

要素	说明
标题	一眼能看懂是什么问题，见 3.6 的公式
环境	机型、系统版本、游戏版本号、账号、服务器。缺一项都可能导致无法复现
前置条件	复现前必须满足的状态，如“背包已满” “等级达到 30”
复现步骤	编号逐步写，每步一个动作，不合并
预期结果	依据是什么（策划案第几节、常理、同类功能表现）
实际结果	客观描述，不加判断
复现率	必现、高概率、低概率、偶现
附件	截图、录屏、日志片段、抓包文件
严重程度	S1 至 S4
优先级	高、中、低

3.6 标题写法

公式：`[模块] + [前置条件] + [操作] + [实际结果] + [复现率]`

不合格	合格
商城有问题	商城-钻石不足时点击购买月卡，提示购买成功但未扣款也未发货（必现）
背包满了买不了东西	商城-背包已满时购买道具，扣款成功但道具未到账且无任何提示（必现）
有时候会闪退	战斗-释放群体技能时有约 30% 概率客户端闪退，附崩溃日志与录屏（高概率）

3.7 复现率怎么判定

- **必现**：按步骤 100% 出现
- **高概率**：多次尝试中出现超过一半
- **低概率**：多次尝试中出现不到一成
- **偶现**：无法稳定复现，仅有截图或日志佐证

> **低概率缺陷必须提，不能因为复现不了就不提。**新人最常见的错误是“我复现不了，可能是偶发，算了”。但偶现的崩溃往往是线上事故的源头，而且开发拿到日志可能十分钟就定位了。**正确做法：提单 + 标注复现率 + 附上抓到的日志，把判断权交给开发。**你放弃的那条，可能就是线上事故的那条。

第四部分・练习题与答案

先自己写，再看答案。每题限时 10 分钟，写不出来说明对应的方法还没掌握。

练习 1・等价类与边界值

某游戏每日签到奖励：连续签到 1 至 6 天为普通档，第 7 天为大奖档，第 8 天起重置回第 1 天。请设计测试点，并说明哪些属于边界值。

▼ **参考答案**

等价类：有效类为 1 至 7 天的连续签到；无效类为连续天数小于 1（不可能达到，但要验证异常数据）与大于 7（应被重置逻辑拦截）。

边界值（核心）：

天数	类型	验证重点
0	零点	首次签到前的状态，不应报错
1	下边界	首次签到，奖励为普通档第 1 档
6	内点	第 7 天前的最后一天，奖励仍为普通档
7	上边界	**关键**：必须发放大奖，且是“第 7 天”而非“第 6 天”
8	离点	**关键**：应重置为第 1 天，发放普通档第 1 档

容易漏的补充点：跨天的时间边界（23:59:59 与 00:00:01 之间签到算哪一天）、断签后是否重置、补签卡对连续天数的影响、跨时区或服务器时间与本地时间不一致时的判定。

判定说明：7 和 8 是这道题的核心。多数人只测到 7，漏掉 8 的重置逻辑，而重置写错（比如第 8 天仍发大奖）是这类功能最典型的缺陷。

练习 2・判定表

公会申请加入规则：玩家等级不低于 20 级、且未加入任何公会、且目标公会人数未满。三个条件共同决定“能否提交申请”。请用判定表列出全部规则。

▼ **参考答案**

三个条件各两个取值，共 8 条规则：

规则	等级 ≥ 20	未加入公会	公会未满	结果
R1	是	是	是	允许提交申请
R2	是	是	否	拒绝，提示公会人数已满
R3	是	否	是	拒绝，提示已加入其他公会
R4	是	否	否	拒绝，**提示优先级需确认**
R5	否	是	是	拒绝，提示等级不足
R6	否	是	否	拒绝，**提示优先级需确认**
R7	否	否	是	拒绝，**提示优先级需确认**
R8	否	否	否	拒绝，**提示优先级需确认**

这道题真正的考点不是填表，而是 R4、R6、R7、R8。当多个条件同时不满足时，应该提示哪一条？策划案通常没写。这四种情况必须去问策划确认，否则开发按自己的理解实现，测试按自己的理解验证，最后必然产生争议。

这就是判定表的第二重价值：它不只用来设计用例，还用来暴露需求文档里的空白。

练习 3 • 状态迁移

好友申请的状态：待处理、已同意、已拒绝、已过期（7 天未处理）。请列出迁移矩阵，并指出哪些非法迁移必须测。

▼ 参考答案

起始 \ 事件	同意	拒绝	超时 7 天	重复申请
待处理	已同意	已拒绝	已过期	✗
已同意	✗	✗	✗	✗
已拒绝	✗	✗	✗	待处理（重新申请）
已过期	✗	✗	✗	待处理（重新申请）

必须重点测的非法迁移：

- 已同意的申请再次点击同意——是否会重复添加好友、重复发送通知
- 已拒绝的申请点击同意——是否会被错误地通过
- 已过期的申请点击同意——过期状态是否真的失效
- 双方同时互发申请——是否会产生两条好友关系
- 同意后立即删除好友，再点击同意——状态是否回到一致

核心思路：非法迁移不是“用户不可能这样操作”，而是“用户通过多端、网络重放、界面未刷新等手段完全可以触发”。后端必须做状态校验，不能只靠前端隐藏按钮。

练习 4 • 场景法

玩家首次进入游戏的新手引导流程，请列出基本流，并写出至少 4 条备选流。

▼ **参考答案**

基本流：启动游戏 → 加载资源 → 创建角色 → 播放开场剧情 → 完成教学战斗 → 领取新手礼包 → 进入主城 → 引导结束。

备选流：

场景	验证重点
引导中途杀进程	重进后从哪一步继续？能否跳过已看剧情？奖励会不会重复发放
引导中途断网	重连后状态一致，不卡在中间步骤
引导中途切后台超过 5 分钟	回来后进度保留，不从头开始
引导中点击“跳过”	能否跳过、跳过后的奖励是否照发、跳过后功能是否解锁
重复进入引导（已完成的账号）	不应再次触发，奖励不可重复领取
引导中收到好友消息或系统弹窗	弹窗关闭后能回到原步骤，不丢失状态

为什么备选流价值更高：基本流只要跑通一次，几乎不会出问题。而“杀进程后奖励重复发放”这类缺陷，直接对应可被刷取的资源，属于必须在上线前拦住的 S1 级问题。

练习 5 • 严重程度与优先级判定

以下四个缺陷，请分别判定严重程度与优先级，并说明理由。

编号	缺陷描述
a	战斗中使用某技能有约 1% 概率导致客户端闪退
b	商城“限时礼包”倒计时显示 00:00:00，但实际仍可购买
c	设置界面的音量滑块拖到最左端，仍有声音
d	排行榜第 11 名玩家的头像显示为默认头像

▼ **参考答案**

编号	严重	优先级	理由
a	S1	高	崩溃属于致命级。虽然概率仅 1%，但战斗是核心玩法且高频触发，实际影响人数大。必须查崩溃日志定位，不能因“偶现”搁置
b	S2	高	涉及收入与运营规则。倒计时归零仍可购买，意味着活动无法正常结束，可能造成超卖或资损。虽不崩溃，但商业影响直接
c	S3	低	非核心功能异常，不影响游戏进行。属于体验瑕疵，可排入常规版本
d	S3	低	显示问题，不影响功能。但需注意：如果原因是“第 11 名之后头像接口未返回”，则可能影响更大范围，应进一步确认影响面

这道题的陷阱在 b 和 d。

b 看起来只是“倒计时不准”，很容易被判为 S3 低优先级，但它背后是活动状态机没关闭，涉及真实资金，必须按高优先级处理。**判断严重程度不能只看表面现象，要看它背后失效的是什么机制。**

d 看起来是无害的显示问题，但如果根因是接口分页或数据缺失，实际影响可能是“第 11 名之后全部异常”。**提缺陷时要顺手确认影响范围，这比单纯报现象更有价值。**

下一步

做完这五道题，你应该能感受到一件事：**方法本身很简单，难的是面对一个陌生功能时，知道该从哪个角度切进去。**

建议的巩固方式：拿任意一款手游，任选一个系统（背包、任务、商城、好友），用五种方法各推导一遍，统计每种方法产出多少条用例。做完三个系统，这套思维方式就固化了。

本手册为学习材料，案例中的规则设定为教学假设，不代表任何具体产品的实际逻辑。实际工作中请以策划案与需求文档为准。